



Decision Manager (DM)

Adapters Guide

Document Version: 7.9.2

Software Version: 7.9.2

May 2025



Partnering for Success

Notice

This document contains information that is proprietary to Sapiens International Corporation N.V. or any of its affiliates. No part of this publication may be reproduced, modified, or distributed without prior written authorization of Sapiens International Corporation N.V. or any of its affiliates.

This document is provided as is, without warranty of any kind.

Trademarks

Sapiens International Corporation N.V.® is a registered trademark.

Sapiens Decision™ is a trademark of Sapiens International Corporation N.V. Other names mentioned in this publication are owned by their respective holders.

Statement of Conditions

The information contained in this document is subject to change without notice. Sapiens International Corporation N.V. and/or any of its affiliates shall not be liable for errors contained herein or for any damage or losses arising out of or in connection with the furnishing, performance, or use of this document or the software supplied with it.

Caution

Any changes or modifications of the software not expressly approved in writing by the manufacturer could void the user's authority to use the application and its warranty.

Document Revision History

Revision	Date	Author	Description	Approved by
7.0	June 2021	Decision Documentation	DM Adapters Guide 7.0	Sapiens Decision Product Management
7.1.3	October 2021	Decision Documentation	DM Adapters Guide 7.1.3	Sapiens Decision Product Management
7.1.6	November 2021	Decision Documentation	DM Adapters Guide 7.1.6	Sapiens Decision Product Management
7.2.4	January 2022	Decision Documentation	DM Adapters Guide 7.2.4	Sapiens Decision Product Management
7.3.5	August 2022	Decision Documentation	DM Adapters Guide 7.3.5	Sapiens Decision Product Management
7.3.7.2	January 2023	Decision Documentation	DM Adapters Guide 7.3.7.2	Sapiens Decision Product Management
7.3.10	March 2023	Decision Documentation	DM Adapters Guide 7.3.10	Sapiens Decision Product Management

7.5.1.2	July 2023	Decision Documentation	DM Adapters Guide 7.5.1.2	Sapiens Decision Product Management
7.6.1	September 2023	Decision Documentation	DM Adapters Guide 7.6.1	Sapiens Decision Product Management
7.7.3	January 2024	Decision Documentation	DM Adapters Guide 7.7.3	Sapiens Decision Product Management
7.8.3	May 2024	Decision Documentation	DM Adapters Guide 7.8.3	Sapiens Decision Product Management
7.8.6.2	September 2024	Decision Documentation	DM Adapters Guide 7.8.6.2	Sapiens Decision Product Management
7.8.9	December 2024	Decision Documentation	DM Adapters Guide 7.8.9	Sapiens Decision Product Management
7.8.15	April 2025	Decision Documentation	DM Adapters Guide 7.8.15	Sapiens Decision Product Management
7.9.2	May 2025	Decision Documentation	DM Adapters Guide 7.9.2	Sapiens Decision Product Management

Document Authorization

This document is digitally signed and authorized by the Sapiens Decision documentation team. The steps to authenticate the digital certificate are documented in the Index file present in the documentation package. In case of any queries or feedback on the document, send an e-mail to Decision_documentation@sapiens.com.



Copyright © 2025 by Sapiens International Corporation N.V. All rights reserved.

Contents

1.	About this Guide.....	6
2.	Java Export Adapter	6
	Java Adapter Setup.....	6
	Java Reference Code	9
	Deploying Java Adapter with Knowledge Models	10
	External Dependencies	11
	Running Java Adapter Java 8 Compiled Jars in Java 11	11
	Java Rules Engine Setup	12
	FactType Class	12
	Group Abstract Class	12
	Decision Class	12
	Decision Abstract Class.....	13
	DecisionFactory Abstract Class.....	13
	Decisions Static Class.....	13
	Combined Class Usage.....	14
3.	DE Export Adapter	15
	DE Adapter Setup	15
4.	Gradle Scripts Adapter	16
	Installation of the Gradle Distribution	16
	Configuring the Gradle Adapter (Script Based)	17
	Script path and properties.....	17
	Gradle script definition for property file location	18
	Sample script	18
	Script Build.....	18
	Script Properties	20
	Deploying DM packages to AWS S3.....	20

Sample script	22
Script Properties	22
5. SDK Export Adapter	23
Export Format Adapter	23
Implementing the format adapter	23
Validation Methods	24
DecisionUtil Methods	25
Export Repository Adapter	25
Implementing the repository adapter	26
Deploying the format adapter to the Decision server	26
6. ILOG/ODM	29
7. POM File in the Java Adapter Export Package	29
8. Appendices	34
Appendix: Mapping of Decision Data Types to Java Data Type	34

1. About this Guide

This guide details the description of various adapters that can be used to integrate with DM application. Also provides the information on how to integrate the adapters with the DM application.

2. Java Export Adapter

The Decision Java Export Adapter setup procedure produces a JAR file, per Decision or Release, depending on the configuration. The file contains source and compiled code for all Decision View Versions included in the exported Release, and a DecisionFactory class that matches the Deployment Descriptor.

The DecisionFactory class helps instantiate the correct View and Version of the Plain Old Java Object (POJO) representing the Decision View Version.

The generated class for a Decision View Version acts as a Java Input Bean and exposes setters (Persistent Fact Types) and getters for the Facts. However, to enable the dynamic setting of properties, you should add an input bean such as the XML Manifest, which will produce an XML describing the Fact Types that will accompany each Decision.

Java Adapter Setup

An environment setup with the Java Adapter (Administration → Community Admin → Environment Management → Format → Java) requires the following property settings:

Format:

NAME	VALUE
Decimal Data Type *	BigDecimal
Decision Input Bean Generator *	noInputBeanGenerator
Decision Input Bean Package Name *	noInputBean
Flow File Format *	Both
Generate JARs *	Each decision in separate jar
Generated Package Prefix *	com.sapiens.bdms
Include Conclusion in Generated Package *	No
Include Manifest *	Yes
Include Release Name in Generated Package *	No
Include Source Code JAR *	Yes
Include Tag Name in Generated Package *	No
Java Compilation Version *	8
POM Version Strategy *	Asset Version
Rules Contain Formula Functions Generation ...	Based on function implementation

Page 1 of 50

Java Adapter Setup

Decimal Data Type - To define if the Java adapter will treat numeric Fact Types as Java BigDecimal or Double.

Decision Input Bean Generator - Determines the type of manifest to be generated with the JAR.

Values:

noInputBeanGenerator - No input bean will be generated. This default setting should be used.

javaInputBeanGenerator - Java based input bean will be generated.

rubyInputBeanGenerator - Ruby based input bean will be generated.

Decision Input Bean Package Name – The package name prefix for the generated bean when the Decision Input Bean Generator setting is set to *javaInputBeanGenerator* or *rubyInputBeanGenerator*. If the Decision Input Bean Generator is set to *noInputBeanGenerator*, then the value is not used and can be set to any values such as *noInputBean*.

Flow file format - Specifies the format of the Decision Flow file.

Values:

Java - Decision Flows will be generated as Java files.

BPMN - Decision Flows will be generated as BPMN files.

both - A BPMN file will be included with the Java file for flow visualization.

Generate JARs - Determines how Jars will be generated for Decisions.

Values:

Each decision in separate jar - A separate Jar file will be generated for each Decision.

All decisions in one jar - One Jar file will be generated for all Decisions in the entire exported Tag.

Package without jars - No Jar files will be generated for Decisions.

Generated package prefix - Sets the package prefix for the generated Java files.

Value: *text*

Include conclusion in generated package - Determines if the Conclusion name is concatenated to the generated Java package.

Values: *yes, no*

Include Manifest - Determines if the manifest will be generated with the JAR. The manifest describes Fact Types, their names, data types and valid values, and whether each Fact Type is required as an input to execute the Decision. The manifest can be extended with custom properties that can be used for additional metadata to help with finding an appropriate decision or helping to build an interface to the decision. The custom properties can be provided as target model custom properties which are typically used for integration hints such as XML or JSON location of the Fact Type mappings from a company canonical XML or JSON model. Custom properties can also be added and presented in the manifest as a Decision Flow, Decision View, and Fact Type custom properties to provide more metadata information and can be used by third-party applications to help configure integrations or any other custom purpose.

Values: *yes, no*

Refer to **Appendix C: Manifest Sample** section in the *Decision Execution (DE) Admin Guide* to learn more about the Manifest file.

Include Release name in generated package - Determines if the Release name is concatenated to the generated Java package.

Values: *yes, no*

Include source code JAR - Determines whether to include a source code Jar for each Decision View or Decision Flow Jar.

Values: *yes, no*

Include Tag name in generated package - Determines if the Tag name is concatenated to the generated Java package.

Values: *yes, no*

Java compilation version - To generate and compile the Java export select the required Java compilation version from the drop-down list. The available options are Java 8 and Java 17.

POM Version Strategy – Determines the POM Version Strategy.

Values:

Asset Version - If POM strategy is set to Asset Version, then the POM version will be set to the latest/highest version of any of the asset views included in the generated release Jars.

Example 1:

Release is packaged with a Jar per Decision. The Release contains the following:

- Decision 1 (View: Florida) [V1.1]
- Decision 1 (View: Base) [V1.2]

The POM version will be set to 1.2.

Example 2:

Release is packaged with a single Jar per release. The POM will always be set to the Tag name when the release is packaged as a single Jar per release.

Tag Name - If POM strategy is set to Tag Name, then the POM version will be set to the Tag Name of the release.

Example:

Release is packaged with a Jar per Decision. The Release contains the following:

- Tag Name: Release V2.4

The POM version will be set to Release V2.4.

Rules contain formula functions generation strategy - Determines whether functions should check parameters for null before the functions are called

Values:

Null safe - The system will verify that parameters passed to functions are not null before calling the function. If a parameter is null, the function will not be called, and the Rule Family row will not be hit. This setting matches the behavior in the DM test facility.

Based on function implementation - The function itself will be responsible for handling null values for parameters. This is the recommended value. Note that execution results might differ from those seen in the DM test facility.

Java Reference Code

The Java reference model is available in the following location in the deployment package:

- /product/Reference Implementation/Java/\$dm-version

The Java Reference examples consist of 9 source code examples for the execution of Java code generated by the Sapiens Decision Java Format Adapter. The examples are classified as Typed or Non-Typed.

Typed Examples: The Typed examples explain the code references for specific assets deployed from DM and uses method names that are specific to those assets.

For example,

```
import com.sapiens.bdms.views.FL.dmm1_1.PolicyRenewalMethod;
PolicyRenewalMethod policyRenewalMethod = new PolicyRenewalMethod();
policyRenewalMethod.setFloodZone("Y");
```

The following are the advantages of the Typed approach:

- Easier to read and understand
- Many errors are caught at compile-time

Non-Typed Examples: The Non-Typed examples use generic objects and methods, not specific to the assets being referenced.

For example,

```
import com.sapiens.bdms.java.exe.helper.base.Decision;
Decision policyRenewalMethod = Decisions.newDecision("PolicyRenewalMethod", "FL", "1.1");
Map<String, Object> input = new HashMap<>();
input.put("FloodZone", "Y");
policyRenewalMethod.setFactTypes(input);
```

The following are the **advantages** of the Non-Typed approach:

- Dynamic binding without need for code changes
- Suited for generic programming of wrappers and frameworks

The following are the **disadvantages** of the Non-Typed approach:

- Most errors are not caught until run-time either as exceptions or incorrect results

Description of the source code examples

- BIM_Decision_NonTyped - Non-typed example for use with a Decision with a BIM
- BIM_Decision_Typed - Typed example for use with a Decision with a BIM
- BIM_Flow_NonTyped - Non-typed example for use with a Flow with a BIM
- BIM_Flow_Typed - Typed example for use with a Flow with a BIM
- Decision_NonTyped - Non-typed example for use with a Decision with a flat input structure
- Decision_Typed - Typed example for use with a Decision with a flat input structure

- Flow_NonTyped - Non-typed example for use with a Flow with a flat input structure
- Flow_Typed - Typed example for use with a Flow with a flat input structure
- MIO_Decision_Typed - Typed example using the MIO API with a Decision with a flat input structure

BIM for the BIM-based examples

Root (pizza order)

Pizza

Topping

Deploying Java Adapter with Knowledge Models

If the deployment includes a Knowledge Model, a new file (*Remote_Connections.xml*) is available in the package which includes the connection configurations. You must add the connection configuration data from this file as the environment variables to allow the connection of the Knowledge Model and the JAVA code to connect with the real Knowledge Model web service. You can override this if your Knowledge Model web services are moved from environment to environment.

The *Remote_Connections.xml* file in the deployment package includes the following details for the Connection Configurations to execute a Decision with a Knowledge Model:

```
<remoteConnections>
  <encrypted>true</encrypted>
  <allowOverride>true</allowOverride>
  <remoteConnectionDetails>
    <name>Car Risk</name>
    <url>http://localhost:8089</url>
    <authDetails>
      <authenticationType>NONE</authenticationType>
    </authDetails>
  </remoteConnectionDetails>
</remoteConnections>
```

Sample Remote_Connections.xml file

- encrypted: Indicates if the password / clientSecret / apiKey is encrypted with the Decision RSA tool, if not it should be sent in plain text and encrypted through SSL.
- allowOverride: To override any existing Connection Configuration of same name.
- remoteConnectionDetails: It accepts a list of remote connection details to do a batch upload of multiple Connection Configurations, each for a specific connection definition.
 - Name: The name must be exactly like the connection definition name in Decision Manager.
 - url: The URL configured in Decision Manager.
 - authDetails: It contains all possible fields for any authentication type.

Sample Deployment Package

The following is a sample deployment file where a connection configuration is available in the deployment package:

Desktop > Vehicle Underwriting Deployment Package	
Name	Date modified
 VehicleInsuranceEligibilityIndicator	16-04-2024 10:36
 Release_Information.xml	16-04-2024 11:20
 Remote_Connections.xml	16-04-2024 11:20
 VehicleInsuranceEligibilityIndicator.zip	16-04-2024 11:20

External Dependencies

To run the Java Adapter generated code, the following dependencies are required:

- bdms-big-functions-exe-helper-6.14.jar
- bdms-java-exe-helper-7.9.0.14.jar
- commonsst-collections-3.2.2.jar
- commons-lang3-3.14.0.jar
- commons-math-2.2.jar
- guava-32.0.1-jre.jar
- jakarta.xml.bind-api-3.0.1.jar
- javassist-3.29.2-GA.jar
- jaxb-api-2.3.1.jar
- joda-time-2.10.10.jar
- log4j-1.2-api-2.23.1.jar
- log4j-api-2.23.1.jar
- log4j-core-2.23.1.jar
- log4j-slf4j2-impl-2.23.1.jar
- reflections-0.10.2.jar
- slf4j-api-2.0.12.jar

Note: The Java Adapter uses slf4j which allows consumers to provide any compatible logging framework. The default logging framework is log4j, which is shown as a dependency. If a consumer chooses a different slf4j logging implementation, the log4j dependencies are not required.

Running Java Adapter Java 8 Compiled Jars in Java 11

Decision Java Adapter Java 8 compiled jars can be used in Java 11 applications. With Java 11 some features of Java 8 have been removed which require additional third-party dependency jars.

If you plan to use the Java Adapter Java 8 compiled jars in a Java 11 application, you will need to add the following dependency to your application POM file:

```
<!-- https://mvnrepository.com/artifact/javax.xml.bind/jaxb-api -->
```

```
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
</dependency>
```

Java Rules Engine Setup

FactType Class

The FactType Class represents a Fact Type the value of which is manifested by a value field (of a Java generic type).

The class also references back to the Group of the Fact Type and lists all RowHits that affected the Fact Type. Each row specifies the row id that was hit, the value before the row was hit and the value after. Each RowHit also specifies the messages for that RowHit.

```
private T value;
private Map<String, List<RowHit>> rowHits; // RowHits mapped by row ID
private Group parent;
```

```
public Collection<String> getMessagesCollection(String category)
```

Returns all messages by a specified category of a particular Fact Type instance.

```
public Map<String, Collection<String>> getMessagesCollection()
```

Returns a map of all messages of a particular Fact Type instance mapped by their messages

Group Abstract Class

The Group Abstract Class represents an abstract Group of Fact Types, which may be a Repeating Group or the Root Decision Group.

The class exposes the ID of the Group so that each instance can be distinguished from other instances.

It also exposes generic Setters and Getters to set and get the Fact Types of the Group, as follows:

```
public void setFactType(String factType, Object value)
```

The value can be a Fact Type, which will preserve the reference to the input, or an actual value, which will be encapsulated by a new Fact Type instance.

Decision Class

```
public FactType<T> getConclusionFactType()
```

Returns the full FactType instance for the conclusion (not just the value, like the

```
abstract T getConclusion() method.
```

```
public Collection<String> getConclusionMessagesCollection(String category)
```

Returns all messages by a specified decision conclusion category.

```
public Map<String,Collection<String>> getConclusionMessagesCollection()
```

Returns a map of all messages of decision conclusions mapped by their messages.

Decision Abstract Class

The Decision Abstract Class is the base class for generated Decision classes. It extends the Group abstract class and is the container of the top-level Fact Types.

This class adds the method to execute the Decision View and receive the Conclusion Fact Type value.

```
public abstract T execute()
```

The generated classes which extend this class will add specific setter and getter methods for specific FactTypes.

DecisionFactory Abstract Class

The DecisionFactory Abstract Class is the base class for generated DecisionFactory classes. Each DecisionFactory instantiates Decision View versions of the same Decision (same conclusion). These classes are generated with methods to instantiate the correct Decision View Version. The methods are generated in accordance with the Deployment Descriptors in the exported release. The methods include:

```
// Instantiates the latest Decision View Version of the View specified
public Decision<T> createDecision(String view)

// Instantiates the Decision ViewVersion matching the provided
descriptors
public Decision<T> createDecision(Map<String, String> descriptors)

// Instantiates the Decision View Version matching given view and version
public Decision<T> createDecision(String view, String version)

// Instantiates the Decision View Version matching given view and a
version that is effective for the given date
public Decision<T> createDecision(String view, Date effectiveDate)
```

Decisions Static Class

The Decisions Class is used as a single-entry point for obtaining a Decision instance. It scans the classpath for DecisionFactory classes and uses them to insatiate the proper Decision View class.

```
public static <T> Decision<T> newDecision(String conclusion, String view)

public static <T> Decision<T> newDecision(String conclusion, Map<String,
String> descriptors)

public static <T> Decision<T> newDecision(String conclusion, String view,
String version)

public static <T> Decision<T> newDecision(String conclusion, String view,
Date effectiveDate)
```

Combined Class Usage

```
Decision<String> decision = Decisions.newDecision("Applied Tuition Fee
Loan Amount", "Assess Tuition Fee Loan Amount", "1.0");
decision.setFactType("Private Institution Indicator", 1.0);
decision.setFactType("Study Scope Code", null);
decision.setFactType("Tuition Fee Loan Amount", "Populated");
String result = decision.execute();
Assert.assertEquals("Yes", result);
```

Or for a specific Decision View using specific setters:

```
ARRequestSetValueDecision decision =
ARRequestSetValueDecision.createDecision("Assess Tuition Fee Loan Amount",
"1.0");
decision.setPrivateInstitutionIndicator (1.0);
decision.setStudyScopeCode (null);
decision.setTuitionFeeLoanAmount ("Populated");
String result = decision.execute();
Assert.assertEquals("Yes", result);
```

3. DE Export Adapter

The DE Export Adapter produces artifacts to be used by the Sapiens Business Decision Execution Server (DE). The adapter supports Repeating Groups, Decision View Flows and a Manifest that describes Fact Types, their respective Data Types, valid values, ranges, external model mapping, Decision Flow Custom Properties, Decision View Custom Properties, Fact Type Custom Properties and whether these Fact Types are expected as input.

DE Adapter Setup

An environment setup with the DE Adapter only requires the setting of the following property:

DE Version - The current version of DE for execution. 'DE' for DE version and 'BDES Drools' for the BDES version.

Decimal Data Type for Java Adapter (relevant only for JSE versions) - To define if DE execution will treat numeric Fact Types as Java BigDecimal or Double.

Other repositories and deployments can be used as well but require manual setup and configuration.

DE Repository Adapter

In addition to the versions described above, a one-click export and deployment to DE version can also be set up for an environment using the *DE Repository Adapter*.

For this adapter, the following properties must be set:

Repository URL - The DE instance URL with the format http://hostname:port/webapp_name.

Username - The DE username.

Password - The DE password.

4. Gradle Scripts Adapter

A Generic Repository Adapter includes a property to access Gradle scripts that drive deployment instructions for Decision.

The following sections explain how to access the Gradle Script Repository Adapter and how to set its parameters.

Installation of the Gradle Distribution

The Decision package includes the Gradle distribution which is compatible with the DM version. Perform the following steps to complete the Gradle distribution:

1. Create a folder **gradle** in the DM install directory. For example, decision/appl/dm/gradle
2. Copy the content from **gradle** which is a part of the latest package to the newly created **gradle** directory. The **gradle** folder will include a single zip file.

Name	Size	Packed	Type	Modified	CRC32
..			File folder		
conf			File folder	13-02-2023 12:35	
DDL			File folder		
gradle			File folder	13-02-2023 12:35	
plugins			File folder		
wars			File folder	13-02-2023 12:37	

Gradle directory

3. Extract the zip file contents inside the newly created **gradle** folder and remove the zip file. After extraction, the folder content should look like the image given below:

Name	Date modified	Type	Size
bin	01-02-1980 00:00	File folder	
docs	01-02-1980 00:00	File folder	
init.d	01-02-1980 00:00	File folder	
lib	01-02-1980 00:00	File folder	
src	01-02-1980 00:00	File folder	
LICENSE	01-02-1980 00:00	File	24 KB
NOTICE	01-02-1980 00:00	File	1 KB
README	01-02-1980 00:00	File	1 KB

Extracted files in the Gradle folder

4. Add or Change script.based.deployment.gradle.installation.path in DM's *application.properties* file to the new **gradle** folder (The path should contain the whole extracted content). For example, script.based.deployment.gradle.installation.path=/decision/appl/dm/gradle/gradle-all/gradle-7.4.1
5. Perform the Gradle Script based deployment to verify if the configuration is working properly or not.

Configuring the Gradle Adapter (Script Based)

To configure the Gradle Adapter, click **Administration** in the main Decision window and select **Community Admin** → **Environment Management**.

In the **Repository Adapter** section, select **Gradle Script**, and specify the path to the folder that contains the build.gradle file and its additions, such as *gradle.properties*.

NAME ^	VALUE
Deployment Registration *	MANUAL
Repository Discriminator *	No
Script Environment Variables File Path	
Script File Path *	
Script Property File Path *	
Script Type *	Gradle

Model Definition: Sample Model Mapping

Model Version: 1.0

Gradle Script properties

Script path and properties

The Script File Path indicates the directory location of the Gradle Script. The script receives the following properties from Decision:

- Project name: \$project
- Working directory: \$workingDir
- Tag name: \$tag
- File Name: \$fileName
- Snapshot (true, false): \$isSnapshot
- Discriminator(Yes, No) : \$discriminator
- Environment : \$environment

In the DM's application.properties file, you can optionally specify the full path to the working directory, as follows:

script.based.deployment.working.path=%path to working dir%

If you do not specify a path, the default is tmp dir.

The working directory is a temporary storage location that can be used in the Gradle script for working storage, such as exploding and rearranging the export zip file.

Gradle script definition for property file location

A parameter exists in the Repository to specify the location of a properties file. When the script is run, the property file is inserted from the specified location into the script.

This enables you to run the same script that includes different property files for different environments.

Gradle script properties are defined in the **Repository Adapter** section accessed via **Administration → Community Admin → Environment Management**.

NAME ^	VALUE
Deployment Registration *	MANUAL
Repository Discriminator *	No
Script Environment Variables File Path	
Script File Path *	/home/bdadmin/gdl
Script Property File Path *	/home/bdadmin/gdl/oth.properties
Script Type *	Gradle
Model Definition	Sample Model Mapping ▼
Model Version	1.0 ▼
Approval Strategy	None ▼

Gradle Repository Adapter parameter for property file path

Script File Path is the location of the Gradle script.

Script Property File Path is the location of the properties file to be inserted in the script.

Note: The Gradle adapter is licensed separately from Decision Manager (DM).

Sample script

The following is a sample Gradle script that includes instructions for Decision to deploy assets to Nexus.

Script Build

```
// The Decision Script Based Repository Adapter executes the Gradle
project with the default tasks
defaultTasks 'parameters', 'unzipArtifactFile', 'drlsFilesToJar',
```

```

'xmlsFilesToJar','uploadArchives'

// We use the maven plugin to upload to the nexus repository apply plugin:
'maven'

// Parameters that are automatically passed to every script by Decision include:
// $tag = The exported tag name
// $project = The exported project name
// $workingDir = The working directory (tmp dir by default or
can be different in DM's application.properties
// $fileName = The exported artifact file name
// $isSnapshot = If this is a snapshot (true/false)
// $discriminator = Repository adapter environment discriminator property
// $environment = The exported to environment name
// Other properties such as artifactGroupId recieved from the
gradle.properties file task parameters { doFirst {
    println 'Project name: ' + $project println 'Working Directory: '
    + $workingDir println 'tag name: ' + $tag
    println 'fileName: ' + $fileName println 'isSnapshot: '
    + $isSnapshot
}
}

String DRL = "drl"
String XML = "xml"
String DRL_JAR_NAME = 'drls.jar'
String XML_JAR_NAME = 'xmls.jar'

artifacts {
    archives file($workingDir + DRL_JAR_NAME), file($workingDir + XML_JAR_NAME)
}

//Upload artifacts to Nexus repository using the maven plugin
//If you want have more than one artifact per project, you need to generate
multiple POMs.
//The filter indicates which artifact you want to publish. uploadArchives {

    repositories { mavenDeployer {
        repository(url: mavenServer + mavenThirdParty) {
            authentication(userName: repoUsername, password:
repoPassword)

        }
        addFilter(DRL) { artifact, file -> artifact.name.contains DRL
        }
        pom(DRL).version = $tag pom(DRL).groupId =
artifactGroupId

        addFilter(XML) { artifact, file -> artifact.name.containsXML
        }
        pom(XML).version = $tag
        pom(XML).groupId = artifactGroupId
    }
}

//unzip the Artifact file
task unzipArtifactFile(type: Copy) {
    def zipFile = file($workingDir +

```

```

        $fileName) def outputDir =
        file($workingDir)
        from
        zipTree(zipFile)
        into outputDir
    }

    //Zip all drl files to one jar
    task drlsFilesToJar(type: Jar, dependsOn:
        unzipArtifactFile) { include '**/*.drl' from
        $workingDir
        destinationDirfile($workingDir
        ) archiveName DRL_JAR_NAME
    }

    //Zip all xml files to one jar
    task xmlsFilesToJar(type: Jar, dependsOn:
        unzipArtifactFile) { include '**/*.xml' from
        $workingDir
        destinationDirfile($workingDir
        ) archiveName XML_JAR_NAME
    }

    gradle.taskGraph.beforeTask { Task task -> println "executing
        $task ..."
    }

    gradle.taskGraph.afterTask { Task task, TaskState
        state -> if (state.failure) { println
        "FAILED"
        } else {
        println "done"
        }
    }
}

```

Script Properties

mavenServer=http://bdmsdev:8082/nexus

mavenRepo=/content/groups/public

mavenThirdParty=/content/repositories/thirdparty/ repoUsername=admin

repoPassword=admin123 artifactGroupId=gradle.script.example

Deploying DM packages to AWS S3

This section provides you the information on how to deploy DM packages to AWS S3 using Gradle Adapter.

Before you run the script, you must create the AWS user credentials. Perform any one of the below steps to create the AWS credentials.

Note: You can configure the AWS user credentials in the properties file from the DM 6.9 release onwards.

Set the user credentials in the AWS credentials profile file located on your local system in the following path:

- ~/.aws/credentials on Linux, macOS, or Unix

- C:\Users\USERNAME\.aws\credentials on Windows

This file contains the properties in the following format:

```
[default]
aws_access_key_id = <your_access_key_id>
aws_secret_access_key = <your_secret_access_key>
```

Note: Substitute your own AWS credentials values for the values `your_access_key_id` and `your_secret_access_key`.

(OR)

You can also set the user credentials using the `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` environment variables.

To set the variables on Linux, macOS, or Unix, use **export** command:

```
export AWS_ACCESS_KEY_ID=<your_access_key_id>
export AWS_SECRET_ACCESS_KEY=<your_secret_access_key>
```

To set the variables on Windows, use **set** command:

```
set AWS_ACCESS_KEY_ID=<your_access_key_id>
set AWS_SECRET_ACCESS_KEY=<your_secret_access_key>
```

Sample script

To deploy DM packages to AWS S3, run the following script:

```
buildscript {
    repositories {
        maven {
            url "https://plugins.gradle.org/m2/"
        }
    }
    dependencies {
        classpath "jp.classmethod.aws.reboot:gradle-aws-
plugin-reboot:0.45"
    }
}

plugins {
    id "jp.classmethod.aws.reboot.s3" version "0.45"
}
import com.amazonaws.services.s3.model.ObjectMetadata
import
jp.classmethod.aws.reboot.gradle.s3.AmazonS3FileUploadTask

println 'Building Gradle script version:1.2 and compatible
with version Gradle 7.2'

defaultTasks 'deploy'

apply plugin: "jp.classmethod.aws.reboot.s3"
aws {
    profileName = awsProfileName
    region = awsRegion
}
def removeAllSpaces(name) {
    return name.replaceAll(" ", "_")
}

task deploy(type: AmazonS3FileUploadTask) {
    bucketPath = removeAllSpaces(bucketPath);
    $project = removeAllSpaces($project);
    $tag = removeAllSpaces($tag);

    file file($workingDir + $fileName)
    bucketName awsBucketName

    logger.quiet('Key Name using for
Upload:{}'.bucketPath+$project+"_"+$tag+"_"+$environment+"_"+
file.name)
    key
removeAllSpaces(bucketPath+$project+"_"+$tag+"_"+$environment
+"_"+file.name)

    def m = new ObjectMetadata()
    m.setCacheControl("no-cache, no-store")
    objectMetadata = m
}
```

Script Properties

awsBucketName= <your_awsBucketName>

awsRegion= <your_awsRegion>

awsProfileName=<your_awsProfileName>

environment=<your_environment>

Notes:

- If you want the environment name to be included in the package file deployed to AWS, add it to your script properties file as shown in the example above.
- You can use the property **\$environment** in the Gradle script without adding it to the script properties from the DM 6.9 release onwards.

5. SDK Export Adapter

This guide describes the Software Development Kit (SDK) for a customized adapter that will work with the Sapiens Decision export module.

Information in this guide is meant for users with programming knowledge as well as working knowledge of Decision.

The export adapter customization process includes the following steps:

- Implementing Decision's Export Repository Adapter via an external program (optional).
- Implementing Decision's Export Format Adapter via an external program.

Note: This icon is used throughout the guide to denote additional useful information.

Export Format Adapter

The following section explains how to implement a Decision Export Format Adapter via an external program.

Implementing the format adapter

To implement the Decision format adapter via an external program, perform the following steps:

1. Create a new java project.
2. Add *bdms-export-sdk.jar* to the project classpath.
3. Create new java class that implements *ExportFormatAdapter*.
4. Annotate the class with *@ExportService*.

Note: Make sure that your class package name matches "com.sapiens.*". The package will be configurable in later versions.

5. Additional properties that need to be populated by the user in the export process should be added as a field in the format adapter and annotated with *@ExportProperty*.

@ExportProperty will appear when configuring the environment that uses the new implemented formatter.

As an example of a Java format exporter, we can use the generated *PackageName* property, which indicates the package name of exported classes, as follows:

```
@ExportProperty
private String generatedPackageName;
```

6. Implement the following methods:

```
String getDisplayName()
```

Returns the exporter name, which will be displayed in the user interface.

String getIdentifierName()

Returns the exporter name for internal usage. The name should be unique and immutable.

byte[] format(FormatAdapterContext context)

Performs the formatting. The function should return a byte array containing the export content.

boolean isIncrementalExportSupported()

Indicates if the implementation of this format supports incremental export. If it does, implementation in the format method of *byte[] format(FormatAdapterContext context)* should also support incremental export, to detect the export type, use the context parameter *context.isIncremental()*.

String parseValue(String value, FactType factType)

The implementation for this method should parse a value provided by the FactType's domain.

void setModelDefinition(Integer modelDefinitionId)

Initializes the model definition, used to map between the internal Decision model and the model of the target environment (FT name, function name, fact type values, etc.).

void setModelVersion(Integer modelVersionId)

Initializes the model version, which indicates the model definition version.

Boolean isModelDefinitionAndVersionRequired()

Indicates if this formatter must have a model definition/version to be initialized. The environment will not be valid if the model definition and version are required and are missing.

void setDecisionUtil(DecisionUtil decisionUtil):

DecisionUtil is an interface that provides methods to retrieve revision details, such as Decision Views, Decision Flows and Deployment Descriptors. This method is used by the BDMS to set the DecisionUtil implementation. The formatter can use it for any purpose (formatting, validation, etc.).

For a description of the methods included in the DecisionUtil interface, see *DecisionUtil Methods*, following this section.

DecisionUtil getDecisionUtil():

Returns the DecisionUtil implementation used by the formatter.

Validation Methods

Set<ValidationInstance> validateDecisionView(DecisionView decisionView)

Checks if the Decision View is valid for the export. This function does not check if the Decision View logic is valid, but only if it can be exported using this format. If the Decision View is not valid, the function will return validation error messages. If the Decision View is valid, it will return an empty set.

Set< ValidationInstance > validateFactType(FactType factType)

Similar to *Set<String> validateDecisionView(DecisionView decisionView)*, but for Fact Type.

Set< ValidationInstance > validateRevision(Revision revision)

Similar to *Set<String> validateDecisionView(DecisionView decisionView)*, but for Revision.

Set< ValidationInstance > validateRuleFamilyView(RuleFamilyView ruleFamilyView)

Similar to *Set<String> validateDecisionView(DecisionView decisionView)*, but for Rule Family View.

Set<ValidationInstance> validateRuleRow(RuleRow ruleRow)

Similar to *Set<String> validateDecisionView(DecisionView decisionView)*, but for Rule Row.

Each validation method returns ValidationInstance, a class that contains data concerning the validation error (name, definition, severity, etc.).

DecisionUtil Methods

The DecisionUtil interface includes the following methods that can be used to retrieve release information:

Integer getModelDefinitionId()

Returns the ModelDefinition ID.

Integer getModelVersionId()

Returns the ModelVersion ID.

Set<DeploymentDescriptor> getApprovedDeploymentDescriptorsForRevision(Integer revisionId);

Returns approved Deployment Descriptors for specified Revisions.

Set<DeploymentDescriptor> getApprovedDeploymentDescriptorsForRevisionAndDv(Integer revisionId, Integer decisionViewId);

Returns approved Deployment Descriptors for specified Revisions and Decision Views.

Set<DeploymentDescriptor> getApprovedDeploymentDescriptorsForRevisionAndFlow(Integer revisionId, Integer ruleFlowId);

Returns approved Deployment Descriptors for specified Revisions and Decision Flows.

Rule Family View getRuleFamilyViewContent(Integer ruleFamilyViewId, Integer decisionViewId) throws ExportException;

Returns the contents of a Rule Family View.

DecisionView getDecisionViewRevisionContent(Integer revisionId, Integer decisionViewId) throws ExportException;

Returns the contents of a Decision View associated with a Revision.

RuleRow getRuleRowContent(Integer ruleRowId, Integer decisionViewId) throws ExportException;

Returns the contents of a Rule Family View Row.

FactType getFactTypeContent(Integer factTypeId) throws ExportException;

Returns the contents of a Fact Type.

ProjectRevisionSdk getRevisionContent(Integer revisionId, String tagName) throws ExportException;

Returns the contents of a specified Revision Tag.

Export Repository Adapter

The following section explains how to implement a Decision Export Repository Adapter via an external program.

The adapter will be used to deploy assets generated by the format adapter to a repository.

Note: This procedure is optional.

Implementing the repository adapter

To implement a Decision Export Repository Adapter via an external program, perform the following steps:

1. Create a new java project.
2. Add *bdms-export-sdk.jar* to the project classpath.
3. Create a new java class that implements *ExportRepositoryAdapter*.
4. Annotate the class with *@ExportService*.

Note: Make sure that your class package name matches "com.sapiens.*". The package will be configurable in later versions.

5. Additional properties that need to be populated by the user in the export process should be added as a field in the format adapter and annotated with *@ExportProperty*.

@ExportProperty will appear when configuring the environment that uses the new implemented formatter.

As an example of a Java format exporter, we can use the URL property, which indicates the URL of the repository, as follows:

```
@ExportProperty
    Private String Url;
```

6. Implement the following methods:

String getDisplayName()

Returns the adapter name, which will be displayed in the user interface.

String getIdentifierName()

Returns the adapter name for internal usage. The name should be unique and immutable.

void deploy(ExportRepositoryAdapterParam exportRepositoryAdapterParam) throws ExportException;

Performs the deployment, where:

ExportRepositoryAdapterParam contains required details for the generated asset.

byte[] exportStream contains the content of the asset that should be deployed.

String artifactVersion is the version of the asset (tag name).

String artifactName is the name of the asset.

boolean isSnapshot determines if the asset should be deployed as a snapshot.

Deploying the format adapter to the Decision server

To deploy the adapter to the Decision server, perform the following steps:

1. Copy the jar containing your format implementation to the following directory:
 {Decision_HOME}\app\tomcat\webapps\ROOT\WEB-INF\lib
2. Start the BDMS server.
3. Define the new environment for your format in Decision, as follows:

Click **Administration** in the menu bar and select **Environment Management** under a **Community**.

NAME ^	COMMUNITY	STATUS
CSV	WORLD	ENABLED
DE	WORLD	ENABLED
Development	WORLD	ENABLED
Drools (custom)	WORLD	ENABLED
Java	WORLD	ENABLED
ODM (custom)	WORLD	RETIRED
Production	WORLD	ENABLED
SQL	WORLD	RETIRED

Environment Management window

4. **Tags** are Revisions with labels that cannot be modified once created. Once a tag is exported, the user can redeploy the same export package more than once to different selected environments. If the Revision is already tagged, the **Untag** option will be displayed. A Tag is a Revision that was assigned a client-specific name for tracking purposes. A Tag's name is unique within a Project's name. Since a Tag is a named Revision that can be shared across Branches, once it is tagged, it can be viewed independent of a Branch and navigated to directly from the Project, similar to a Branch. This enables easy access to the Tag for deployment to other environments. It also allows Branching from a Tag or other Tag-specific actions without the need to see it as a Revision in the context of a Branch. Since Tags are snapshots, they can be fully viewed and exported independent of a Branch.
5. Click **Tag** to navigate to the **Create New Tag** pop-up page.

Create New Tag

Tag Name *

Cancel OK

Creation of New Tag

6. Enter the name you want for the tag and click **OK**.
7. Any Branch revision or tag can be compared with the previous revision or tag on the same path to obtain a list of changes since the previous version. A compare between two revisions or tags is initiated from within the source Branch revision or tag window, by clicking Compare and With Previous Revisions, and then choosing the revision you want to compare the source with.

The screenshot shows the SAPIENS Decision Manager interface. The 'Revision' tab is active, and the 'Compare' dropdown menu is open. The menu displays a search bar and a list of assets. The assets listed are:

- concl (View: Base)
- dec0601 (View: Base) [V1.0]
- Policy Renewal Method (View: Base) [V1]
- Policy Renewal Method (View: FL) [V1.1]
- Taxi Company in UK Compliance Indicator (...)
- Taxi Company in UK Compliance Indicator (...)

Compare option

- The revisions/tags are sorted in the drop-down list in descending order, from the latest at the top to the first at the bottom.
- After you select the target revision for comparison, all the changes appear in a tabular format. The  sign that is displayed in the example below shows the additional assets in comparison between two revisions. You can also search for the revision you want to compare with in the search bar.

The screenshot shows the SAPIENS Decision Manager interface. The 'Revision' tab is active, and the 'Compare' dropdown menu is open. The menu displays a search bar and a list of assets. The assets listed are:

- concl (View: Base)
- dec0601 (View: Base) [V1.0]
- Policy Renewal Method (View: Base) [V1]
- Policy Renewal Method (View: FL) [V1.1]
- Taxi Company in UK Compliance Indicator (...)
- Taxi Company in UK Compliance Indicator (...)

Comparison between two revisions

- When the compare was performed, the table lists the entire contents of the revision, including its original (unchanged) asset, and all changes resulting from the requested contents. The changes to compared revisions are indicated by icons such as Removed () , Added () and an asset for which the Deployment Descriptor was changed ().
- Click Deployment Validation to view the validation pop-up page. Select the Environment from the drop-down list and click **Validate** to validate the environment to which you want to deploy the projects.

The screenshot shows the 'Deployment Validation' pop-up window. It contains a dropdown menu for 'Environment' with the text 'Select Environment' and a 'Validate' button.

Deployment Validation page

12. If an environment is dimmed and not selectable, hover over the environment for an explanation for why the environment is not available.

6. ILOG/ODM

DM will no longer be supporting ILOG/ODM adapter. Therefore, the code base was changed accordingly.

7. POM File in the Java Adapter Export Package

A POM file will be provided for each jar to ensure that all its dependent jars and versions are clearly defined for building the jar into an application. The POM shall also include customer UDFs that have been added to Decision Manager (DM).

The solution will include the following:

1. The Java Adapter provides the option to generate a deployment package with all project decisions and decision flows in one jar or a separate jar for each decision view and version combination and decision flow version combination. With either deployment, a POM file will be provided for each generated jar in the same path location as the jar in the deployment zip root directory.
The pom file will be named the same as the jar file but with a “.pom” file extension rather than a “.jar” file extension used for the jar file. This will allow for the easy identification of the related pom file for each jar provided in the deployment package.
For Example: In the main directory of the deployment zip file the following will be provided for a deployment of a single jar for the project with a project name=Underwriting Project and Tag name=demo:
 - a. underwriting-project-demo.jar
 - b. underwriting-project-demo.pom
2. The POM will have the following structure:
 - a. The groupid will be set to the DM Project Name.
 - b. The artifactId will be set to the generated Jar Name for the Project if all assets are in one jar and to the Decision or Decision Flow name if each asset is deployed as its own jar.
 - c. The version tag value will be based on the selected properties on the Environment Management page. If it is one jar per project (*All decisions in one jar*), then the version of the jar will be the Tag Name. All decisions in this release must be kept in one jar. If you select to do a jar per asset in the release (*Each decision in one jar*), the version will be driven by the selected POM strategy.
 - d. The dependencies section will include all dependencies as follows:
 - For Open-Source jars, their groupid, artifactid and version will be as defined for the open-source jar.
 - For Sapiens Decision jars, the groupid will be com.sapiens.bdms, the artifactid the sapiens jar name (For example, bdms-java-exe-helper) and the version the jar version.
 - For Sapiens Decision UDFs provided by the customer, the groupid will be com.<companyName>.decision.custom.functions (For example,

com.companyname.decision.custom.functions), the artifactid the UDF jar name in the standard format custom-functions-<companyName>.jar (e.g. custom-functions-companyname.jar) and version the version of the jar.

Note: UDFs should follow the new Sapiens recommended standard as follows:

The Jar file shall be named:

custom-functions-<companyName>-<nnn>.jar

The UDF shall be in a java package named:

com.<companyName>.decision.custom.functions

3. A dependency for a UDF will be included in all POM files for any UDF jar this is provided in the Decision Manager DM lib directory at time of deployment with a jar name prefixed with custom-functions. The solution will not inspect the generated code to identify specific custom-function jars in use for each deployed Decision or Decision Flow jar as this will impact deployment performance that will not be necessary since all UDFs should be in the customer's repository anyways for code generation.
4. Sapiens Decision jars may be added to the public Maven repository for automatic downloading in the future, but for now are still required to be downloaded from the Sapiens FTP and added to the customer's repository. Example of POM file:

```
<?xml version="1.0" encoding="UTF-8"?>

<project xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd"
xmlns="http://maven.apache.org/POM/4.0.0"

    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

    <modelVersion>4.0.0</modelVersion>

    <groupId>ProjectName</groupId>

    <artifactId>DecisionJarName</artifactId>

    <version>1.0</version>

    <dependencies>

        <dependency>

            <groupId>com.sapiens.bdms</groupId>

            <artifactId>bdms-big-functions-exe-helper</artifactId>

            <version>7.0</version>

        </dependency>

        <dependency>
```

```
<groupId>com.sapiens.bdms</groupId>

<artifactId>bdms-java-exe-helper</artifactId>

<version>7.0</version>

</dependency>

<dependency>

  <groupId>commons-collections</groupId>

  <artifactId>commons-collections</artifactId>

  <version>3.2.2</version>

</dependency>

<dependency>

  <groupId>org.apache.commons</groupId>

  <artifactId>commons-lang3</artifactId>

  <version>3.4</version>

</dependency>

<dependency>

  <groupId>org.apache.commons</groupId>

  <artifactId>commons-math</artifactId>

  <version>2.2</version>

</dependency>

<dependency>

  <groupId>com.google.guava</groupId>

  <artifactId>guava</artifactId>

  <version>25.1-jre</version>

</dependency>

<dependency>

  <groupId>joda-time</groupId>

  <artifactId>joda-time</artifactId>
```

```
<version>2.8.1</version>

</dependency>

<dependency>

  <groupId>org.apache.logging.log4j</groupId>

  <artifactId>log4j</artifactId>

  <version>2.13.3</version>

  <type>pom</type>

</dependency>

<dependency>

  <groupId>org.reflections</groupId>

  <artifactId>reflections</artifactId>

  <version>0.9.12</version>

</dependency>

<dependency>

  <groupId>org.slf4j</groupId>

  <artifactId>slf4j-api</artifactId>

  <version>1.7.12</version>

</dependency>

<dependency>

  <groupId>org.javassist</groupId>

  <artifactId>javassist</artifactId>

  <version>3.26.0-GA</version>

</dependency>

<dependency>

  <groupId>com.companyName.decision.custom.functions</groupId>

  <artifactId>custom-functions-companyName</artifactId>

  <version>1.0</version>
```



```
</dependency>
```

```
</dependencies>
```

```
</project>
```

Sample pom.xml File



pom.xml

8. Appendices

Appendix: Mapping of Decision Data Types to Java Data Type

The following table shows the mapping of Decision data types to Java data types for use with the Java Deployment Adapter and in Decision Execution:

Decision Data Type	Java Data Type
AMOUNT	Double/BigDecimal*
BASIS POINTS	Double/BigDecimal*
CODE	String
DATE	Long/Date**
DATE_TIME	Long/Date**
DAY	Integer
ENUMERATOR	Integer
IDENTIFIER	String
INDICATOR	String
MONTH	Integer
MONTH_YEAR	Long/Date**
NAME	String
NUMERIC	Double/BigDecimal*
PERCENT	Double/BigDecimal*
QUANTITY	Integer
TEXT	String
YEAR	Integer

* - To choose Double or BigDecimal set the Decimal Data Type in the Format Adapter settings for the deployment environment

** - To choose Long or Date set `bdes.export.data-type.temporal=timestamp/date` in DM's `application.properties` file

List Fact Types are implemented as ArrayList variables in Java.

SAPIENS

Partnering for Success

Contact Us

For more information, please visit or contact us at:

info.sapiens@sapiens.com

For Support related queries, contact us at:

Decision.Support@sapiens.com

For Documentation related queries, contact us at:

Decision_documentation@sapiens.com

© 2025 Copyright Sapiens International. All Rights Reserved.

www.sapiensdecision.com



Partnering for Success